

LIST AND DICTIONARIES

LIST

Basics

- container is an object that groups related objects together.
- mutable container, size of list can grow or shrink and elements of list can be changed.
- sequence type, left to right positional order indexing starting from 0 to last_index - 1.
- can hold different object types at the same time.

Iteration

```
for variable in my_list:  
    # Loop body statements
```

- Each iteration creates a new variable by binding next element of the list to the name variable.
- Common errors:
 - Trying to access a list with an index that is out of the list's range.

List Operations

Operation	Description	Example
<code>my_list = [1, 2, 3]</code>	Creates a list	<code>[1, 2, 3]</code>
<code>list(iter)</code>	Creates a list	<code>list("123")</code> → <code>['1', '2', '3']</code>
<code>my_list[index]</code>	Gets an element from a list	<code>my_list[1]</code> → <code>2</code>
<code>my_list[start:end]</code>	Gets a new list containing some elements (slice)	<code>[1, 2, 3][1:3]</code> → <code>[2, 3]</code>

Operation	Description	Example
<code>my_list1 + my_list2</code>	Concatenates two lists	<code>[1, 2] + [3] → [1, 2, 3]</code>
<code>my_list[i] = x</code>	Changes the value of the element at index <code>i</code>	<code>[1, 2, 3]</code> then <code>my_list[2] = 9 → [1, 2, 9]</code>
<code>del my_list[i]</code>	Deletes the element at index <code>i</code>	<code>[1, 2, 3]</code> then <code>del my_list[1] → [1, 3]</code>
<code>second_list = first_list[:]</code>	Creates a (shallow) copy of a list	<code>second_list</code> is a new list with the same elements

List Methods

Category	Method	What it does
Adding elements	<code>list.append(x)</code>	Adds <code>x</code> to the end of the list
Adding elements	<code>list.extend(iterable)</code>	Adds all items from <code>iterable</code> to the end of the list
Adding elements	<code>list.insert(i, x)</code>	Inserts <code>x</code> at index <code>i</code> (before the current item at <code>i</code>)
Removing elements	<code>list.remove(x)</code>	Removes the first occurrence of value <code>x</code>
Removing elements	<code>list.pop()</code>	Removes and returns the last item
Removing elements	<code>list.pop(i)</code>	Removes and returns the item at index <code>i</code>
Modifying	<code>list.sort()</code>	Sorts the list in-place (ascending by default)
Modifying	<code>list.reverse()</code>	Reverses the list in-place
Lookup	<code>list.index(x)</code>	Returns the index of the first occurrence of <code>x</code>
Counting	<code>list.count(x)</code>	Returns how many times <code>x</code> occurs in the list

Iteration built-in functions

- `enumerate` → iterates over a list and provides an iteration counter.

```
for index, variable in enumerate(my_list):
```

- `all(list)` → True if every element in list is True (`!= 0`), or if the list is empty.

```
print(all[1, 2, 3,]) = True  
print(all[0, 1, 2]) = False
```

- `any(list)` → True if ANY element in the list is True.

```
print(any[0, 2]) = True  
print(any[0, 0]) = False
```

- `max(list)` → Get the maximum element in the list.
- `min(list)` → Get the minimum element in the list.
- `sum(list)` → Get the sum of all elements in the list.

List nesting

A list can contain another list as an element, this is known as list nesting.

```
my_list = [[10, 20], [30, 40]]
```

```
my_list[0] = [10, 20]    and    my_list[1] = [30, 40]
```

```
my_list[0][0] = 10        my_list[1][0] = 30  
my_list[0][1] = 20        my_list[1][1] = 40
```

TIC TAC TOE

```
tic_tac_toe = [  
    ["X", "0", "X"],  
    print(tic_tac_toe[0][0], tic_tac_toe[0][1], tic_tac_toe  
[0][2]) ->    X 0 X  
    [" ", "X", " "],  
    print(tic_tac_toe[1][0], tic_tac_toe[1][1], tic_tac_toe
```

```
[1][2]) ->      X
      ["0", "0", "X"]
      print(tic_tac_toe[2][0], tic_tac_toe[2][1], tic_tac_toe
[2][2]) ->    0 0 X
]
```

List slicing

Can use slice notation (`my_list[0:2]`) to read multiple elements from a list, creating a new

list that contains only the desired elements. 0 indicates the start and 2 excluded as the end range of

elements to retrieve.

- To include the final element of a list in the slice, specify an end position past the end of list.

```
names = ["Armando", "Tyler", "Zeno"] -> print(names[0:2])
-> Creates new list containing the elements in range -> 0
output: Armando and Tyler
```

- Using negative indices can be used to count backward from the end of the list.
 - Such usage is useful when length of the list is unknown.
 - Similar to `len(my_list_ - 1)`.
- Stride: indicates how many elements are skipped between extracted items in the source list.
 - By default, list stride by 1.

```
nums = [0, 1, 2, 3, 4, 5, 6] -> nums[0, -1, 2] -> Output: [0,
2, 4, 6]
```

- Error handling:

- End position that exceeds the length of the list is treated as the end of the list.
- End position less than the start position, an empty list is produced.

List Operations (slicing)

Slice	Description	Example
<code>list[start:end]</code>	Elements from <code>start</code> up to (but not including) <code>end</code>	<code>nums = [0, 1, 2, 3, 4]</code> <code>nums[0:2] → [0, 1]</code>
<code>list[start:end:stride]</code>	Every <code>stride</code> element from <code>start</code> up to (not including) <code>end</code>	<code>nums[0:-1:2] → [0, 2]</code>
<code>list[start:]</code>	From <code>start</code> to the end	<code>nums[2:] → [2, 3, 4]</code>
<code>list[:end]</code>	From the beginning up to (not including) <code>end</code>	<code>nums[:4] → [0, 1, 2, 3]</code>
<code>list[:]</code>	Copy of the list	<code>nums[:] → [0, 1, 2, 3, 4]</code>
<code>list[len(list)//2 : (len(list)//2) + 1]</code>	Gets the middle element (as a 1-item list)	Example: <code>nums[2:3] → [2]</code>

Loops Modifying Lists

"Programmer often iterates over a list while modifying the elements, such as changing elements values or moving elements positions."

CHANGING ELEMENTS VALUES (LEN() + RANGE())

```
list = [1, 2, 3, 4, 5]
for i in range(len(list)):
    list[i] += 5
```

REMOVING VALUE FROM LIST IF IN ANOTHER LIST

```
for val in copiedList1[:]:
    if val in list2:
        list1.remove(val)
```

- Iterate over copied list instead of actual list.
- If an element is removed while process of iteration, element will move down position, causing pointer to
by pass that element and not checking it.

List comprehension

"Python provides a convenient construct, that iterates over a list, modifies each element and returns new modified list"

```
new_list = [expression for loop_variable_name in iterable]
```

1. An expression component to evaluate for each element in the iterable object.
2. A loop variable component to bind to the current iteration element.
3. An iterable object component to iterate over (list, string, tuple, enumerate, etc.)

```
new_list = [(i + 5) for i in before_list]
```

Conditional List Comprehensions

" List comprehension can be extended with an optional conditional clause that causes the statement to return

a list with only certain elements."

- This syntax will only add an element to the resulting list if the condition evaluates to true.

```
new_list = [expression for name in iterable if condition]
```

```
even_numbers = [i for i in numbers if (i % 2) == 0]
```

- returns num that are even numbers.

List Operations (comprehension)

Goal	Example
Add 10 to every element	<pre>lst = [(i + 10) for i in lst]</pre>
Convert every element to a string	<pre>lst = [str(i) for i in lst]</pre>
Convert user input to a list of integers	<pre>lst = [int(i) for i in input.split()]</pre>
Find the sum of each row in a 2D list	<pre>lst = [sum(row) for row in lst]</pre>
Find the sum of the row with the smallest sum (2D list)	<pre>lst = min([sum(row) for row in lst])</pre>
Using <code>split</code> in a list comprehension to convert strings to ints	<pre>lst = [int(i) for i in other_list.split()]</pre>
Getting input and adding to a list (list of rows)	<pre>for i in range(num_rows): lst.append([int(x) for x in input().split()])</pre>

- List comprehension is not an exact replacement of for loops because list comprehensions create a new list object, whereas the typical for loop modifies an existing list.

Sorting List



"`sort()` performs an in-place rearranging of the list elements, sorting elements from lowest to highest"



`sorted()` : built-in function provides the same sorting functionality as the `list.sort()` but instead creates and returns a new list instead of modifying an existing list.

```
numbers = [int(i) for i in input("Enter numbers: ").split()]\n-> Enter numbers: 20, 10, 50, 40, 30
```

```
#Creates a new list with sorted numbers
sorted_numbers = sorted(numbers)
    -> 10, 20, 30, 40, 50
```

- `sort()` and `sorted()` has a built in optional key argument when comparing strings.
 - `str.lower`, `str.upper`, `str.capitalize`
 - `no_key_sort = sorted(names)`
 - `key_sort = sorted(names, key=str.lower)`
- key argument can be assigned to any function like sorting a list with `max()`
 - `sorted_list = sorted(my_list, key=max)`
- Reverse argument can be set to Boolean value, either True or False. Setting `reverse=True` flips sorting
from lowest to highest, from highest to lowest.

```
sorted([15, 20, 25], reverse=True -> [25, 20, 15]
```

DICTIONARIES

" Container in Python that stores data as key-value pairs (like a dictionary that contains words with definitions "

- Unlike strings, tuples and lists, dictionaries do not store items by position in the same way, they use keys
to access values.
- In Python 3.7 and later, dictionaries keep items in the order they were added.
- The built in `DICT` type is used to create dictionaries in Python.

Example

key		Value

"Bobby"		"A+"
"Alan"		"67"
10		5.0

```
my_dict = {  
    "Bobby": "A+",  
    "Alan": 67  
    10: 5.0  
}
```

WAYS TO CREATE A DICT:

1.) One way to create a dictionary is by using braces { } with key-value pairs inside.

Example:

```
{"Jose": "A+", "Gino": "C-"}
```

This creates a dictionary with two keys:

"Jose" → "A+"

"Gino" → "C-"

Here, each name is a key, and each grade is its value.

2.) The second way to create a dictionary is with dictionary comprehension. It works like list comprehension, but it creates a dictionary instead of a list.

This topic is not covered in this material.

3.) Another way to create a dictionary is by using the built-in dict() function.

You can do this in two ways:

- Using keyword arguments:

```
dict(
    Bobby="805-555-2232",
    Johnny="951-555-0055"
)
```

- Using a list of tuples:

```
dict([
    ("Bobby", "805-555-2232"),
    ("Johnny", "951-555-0055")
])
```

Both of these create the same dictionary.

In real programs, a dictionary is often created first and then filled with data later, such as from user input or a file.

Dictionaries are mutable, which means their contents can be changed after they are created. You can add new entries, change existing ones, or remove them.

COMMON DICT OPERATIONS

Operation	Description	Example
<code>my_dict[key]</code>	Retrieves the value associated with <code>key</code>	<code>armando_grade = my_dict["Armando"]</code>
<code>my_dict[key] = value</code>	Adds a new entry if <code>key</code> doesn't exist; otherwise updates it	<code>my_dict["Armando"] = "A+"</code>
<code>del my_dict[key]</code>	Deletes <code>key</code> (and its value) from the dictionary	<code>del my_dict["Jose"]</code>
<code>key in my_dict</code>	Checks whether <code>key</code> exists in the dictionary	<code>if "Armando" in my_dict:</code> <code>...</code>

Dictionary Methods:

" Dictionary method is a function provided by the dictionary type (dict) that operates on a specific dictionary object.

Dictionary methods can perform useful operations such as: Adding or removing elements, obtaining all the keys

or values in the dictionary, merging dictionaries, etc. "

Method	Description	Example	Output
<code>my_dict.clear()</code>	Removes all items from the dictionary	<pre>my_dict = {"Armando": 1, "Lena": 2} my_dict.clear() print(my_dict)</pre>	→ { }
<code>my_dict.get(key, default)</code>	Reads the value of the key from the dictionary. <i>If the key doesn't exist in the dictionary, then returns default.</i>	<pre>my_dict = {"Armando": 1, "Lena": 2} print(my_dict.get("Lena", "N/A")) print(my_dict.get("Mando", "N/A"))</pre>	→ 2 → N/A
<code>my_dict1.update(my_dict2)</code>	Merges dictionaries. <i>Existing entries in first dictionary are overwritten if the same keys exist in second dictionary.</i>	<pre>my_dict = {"Armado": 1, "Lena": 2} my_dict.update({"Miloh": 3}) print(my_dict)</pre>	→ {'Armando': 1, 'Lena': 2, 'Miloh': 3}
<code>my_dict.pop(key, default)</code>	Removes and returns the key value from the dictionary. <i>If key does not exist, then default is returned.</i>	<pre>my_dict = {"Armado": 1, "Lena": 2} value = my_dict.pop("Lena") print(my_dict)</pre>	→ {'Armando': 1}

Iterating over a dictionary



A ***for loop*** can be used to iterate over a dictionary object, with the loop variable set to a key of an entry in each iteration.

Python interpreter creates a hash of each key.

A Hash is a transformation of the key into unique value that allows the interpreter to perform fast lookup.

```
for key in dictionary: #Loop expression
    # Statements to execute in the loop

# Statements to execute after the loop
```



Dict type also supports useful methods: `items()`, `keys()`, and `values()`, which produce a view object.

A view object provides read-only access to dictionary keys and values.

A program can loop through a view object to get one item at a time.

Based on the method used, it can get a key-value pair, a key, or a value.

A view object also stays updated, so if the dictionary changes after the view object is created, the view object shows those changes too.

```
dict.items() – returns a view object that yields (key, value)
tuples.
dict.keys() – returns a view object that yields dictionary ke
ys.
dict.values() – returns a view object that yields dictionary
values.
```

dict.items()

```
num_calories = dict(Coke=90, Coke_zero=0,  
Pepsi=94)  
for soda, calories in num_calories.items():  
    print(f"{soda}: {calories}")
```

```
Coke: 90  
Coke_zero: 0  
Pepsi: 94
```

dict.keys()

```
num_calories = dict(Coke=90, Coke_zero=0,  
Pepsi=94)  
for soda in num_calories.keys():  
    print(soda)
```

```
Coke  
Coke_zero  
Pepsi
```

dict.values()

```
num_calories = dict(Coke=90, Coke_zero=0,  
Pepsi=94)  
for calories in num_calories.values():  
    print(calories)
```

```
90  
0  
94
```



When a program loops through a view object, it gives one result at a time instead of creating a full list of all keys or values first. *This saves memory.*

→ View objects only produce items when needed, *they do not support indexing.*

Example:

`my_dict.keys()[0]` → ERROR

Fix:

Convert the view object to a list with `list()`, then do operations like indexing or sorting.

```
solar_distances = dict(mars=219.7e6, venus=116.4e6,
jupiter=546e6, pluto=2.95e9)
list_of_distances = list(solar_distances.values())
# Convert view to list

sorted_distance_list = sorted(list_of_distances)
closest = sorted_distance_list[0]
next_closest = sorted_distance_list[1]

print(f"Closest planet is {closest:.4e}")
print(f"Second closest planet is
{next_closest:.4e}")
```

```
Closest planet is
1.1640e+08
Second closest
planet is 2.1970e+08
```



The `dict.items()` method is especially useful because it returns a view object that gives tuples with both the key and value. During each loop iteration, those tuples can be unpacked, like with `enumerate()`, so the loop gets both the key and value without needing extra code.

Dictionary Nesting



A dictionary may contain one or more nested dictionaries, in which the dictionary contains another dictionary as a value.

```
students = {}
students ["Jose"] = {"Grade": "A+", "StudentID": 22321}

print("Jose:")

print(f" Grade: {students ['Jose']['Grade']}")

print(f" ID: {students['Jose']['StudentID']}")
```



Explanation →

- Variable students is created as an empty dictionary.
- Indexing operation adds a new entry with "Jose" → Key and another dictionary as the value.
 - To access data inside the nested dictionary, you use two sets of brackets.
 - students["Jose"]["Grade"] first gets the dictionary stored under "Jose", then gets the value for "Grade" from that dictionary.

Nested dictionaries are a simple but useful way to organize data.

A **Data Structure** is a way of arranging data so it is logical and easy to work with.

Lists and dictionaries are already data structures, but placing them inside each other gives a programmer more flexibility in how information is stored.

Example → A gradebook can use nested dictionaries to organize students and their grades.

```
# Nested dictionary containing student gradebook
grades = {
```

```

"Armando Sarza": {
    "Homeworks": [100, 90, 90],
    "Midterm": 90,
    "Final": 100
},

"Lena Castro": {
    "Homeworks": [100, 100, 100],
    "Midterm": 101,
    "Final": 100
},

"Miloh Callie": {
    "Homeworks": [95, 100, 95],
    "Midterm": 100,
    "Final": 95
},

user_input = input("Enter student name")

while user_input != "exit":
    if user_input in grades:
        # Get values from nested dictionary
        homeworks = grades[user_input]["Homeworks"]
        midterm = grades[user_input]["Midterm"]
        final = grades[user_input]["Final"]

        # Print student information

        # Print homework scores
        for homework, score in enumerate(homeworks):
            print(f"Homework {homework}: {score}")
        # Print midterm score
        print(f"Midterm: {midterm}")
        # Print final score
        print(f"Final: {final}")

```

```
# Compute student total score for all three grades
total_points = sum([i for i in homeworks]) + midterm + final

print(f"Final Grade Percentage: {100 *(total_points / 500.0):.1f}%")

user_input = input("Enter student name: ")
```

COMMAND LINE ARGUMENTS

" Values entered by a user when running a program from a command line. A command line exists in some program

execution environments in which a user can run a program by typing at a command prompt. "

Example

To run a Python program named "myprog.py" with an argument specifying the location of a file named

"myfile1.txt", the user enters the following at the command prompt →

```
> python myprog.py myfile1.txt
```

- The contents of this command line are automatically stored in the list `sys.argv`, which is stored in the standard library `sys` module.
- `sys.argv` consists of one string element for each argument typed on the command line.

When executing a program, the interpreter parses the entire command line to find all sequences of characters

separated by whitespace, storing each as a string within the list variable argv.

```
python myprog.py myfile1.txt -> ["myprog.py", "myfile1.txt"]
                                0          1
```

(Example: how to use command line arguments)

```
-----
-----
import sys                                | > python myprog.p
y Tricia 12                               |
                                           | Hello Tricia.
name = sys.argv[1]                       | 12 is a great a
ge.                                       |
age = int(sys.argv[2])                  |
print(f"Hello {name}.")                 |
print(f"{age} is a great age.\n")       |
-----
-----
```

!! COMMON ERROR !!

Access elements within argv without first checking the length of argv to ensure the user

entered enough arguments, resulting in a `IndexError` being generated.

If the user enters too many arguments, extra arguments will be ignored.

!! GOOD PRACTICE !!

Always check the length of argv at the beginning of the program to ensure that the user

entered the correct number of arguments.

- command-line arguments in argv are strings. If an argument represents a different type like a number,

then the argument needs to be converted using one of the built-in functions such as `int()` or `float()`.

- Command-line arguments may have space in between like (Armando Sarza), use double quotes to ensure this argument is one.

(Checking number of command-line arguments)

```
-----  
-----  
if len(sys.argv) != 3:  
    print("Usage: python myprog.py name age\n")  
    sys.exit(1) # Exit the program, indicating an error with  
1.  
-----  
-----
```